Graphics and BoosterPackDisplay

ECE 2564, Embedded Systems

Fall 2025

The BoosterPack Display

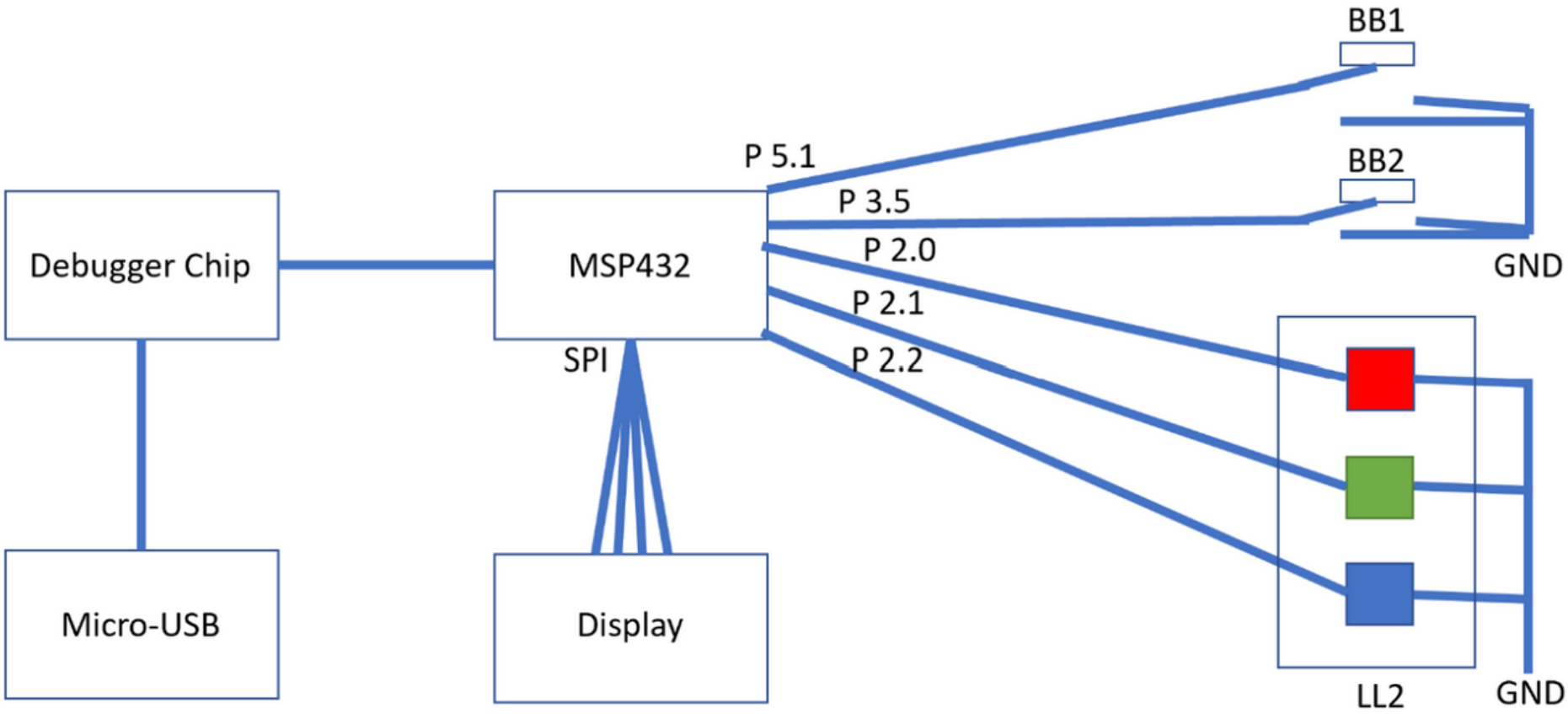
- The display is connected to a peripheral named Serial Peripher Interface (SPI)
- We'll use the SPI now even though we won't study it in detail until the end of the semester
- The name of our display is Crystalfontz
- The display uses a driver library of its own named **GRLIB** (graphics libary)
- We have already used this library in HW2 – it was treated as a "black box"

GRLIB Highlights

This library consists of

- A variety of useful functions for drawing characters and shapes and other graphic-related activities
- Some key structures that are used in these functions
- Some useful macros such as colors

Block Diagram of Typical Display Use Case



Using GRLIB

The most common structure that is used by almost all GRLIB functions is:

```
typedef struct Graphics_Context
{
    int32_t size;  // Size of this structure
    const Graphics_Display *display;  // The screen being drawn on
    Graphics_Rectangle clipRegion;  // The clipping region to be used
    uint32_t foreground; // The color of the primitives
    uint32_t background; // The background color
    const Graphics_Font *font; // The font color for text
} Graphics_Context;
```

We typically only work with the last three members of this structure:

- `foreground`
- `background`
- `font`

Setting up the Display

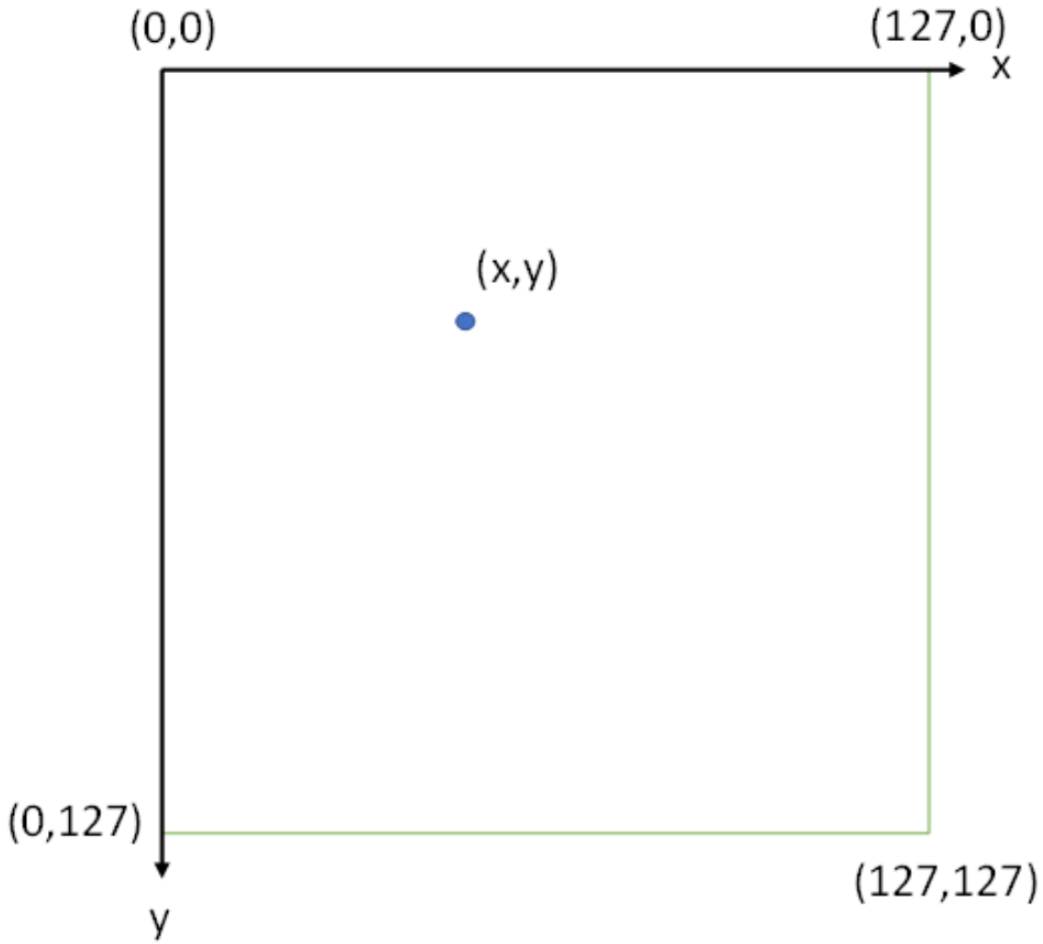
- Like other peripherals, the display needs to be initialized
- Initialization consists of setting up the SPI connection and then initialization of the display itself
- We use these provided initialization functions as black boxes and do not change them for now
- We can change the background and foreground colors and font.

“Black Box use of GRLIB”

Import this repository:
https://github.com/ECE2564-VT/basic_example_graphics

The Display's Coordinate System

The display is 128x128 pixels



Shapes (shown in Foreground color)

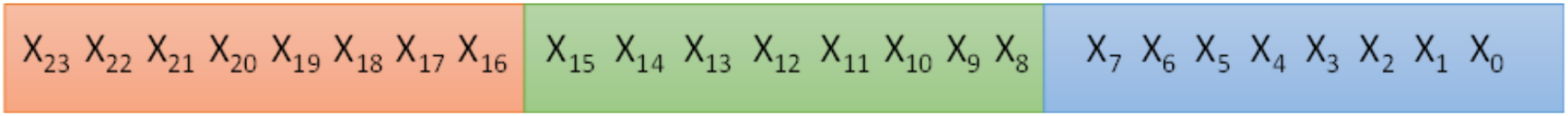
- Rectangles
  - Empty (just the perimeter or frame)
  - Filled (painted over)
- Lines
  - Vertical
  - Horizontal
  - Slanted
- Circles
  - Empty (just the perimeter)
  - Filled (painted over)

Learning to use GRLIB

- Pay careful attention to the header definitions
- READ the documenation in the GRLIB guide
- `CTRL` + mouseclick will show a function's details
- Or hover over a function to see the details

Using Colors

Colors are represented by Red-Green-Blue (RGB) format with 24 bits of data:

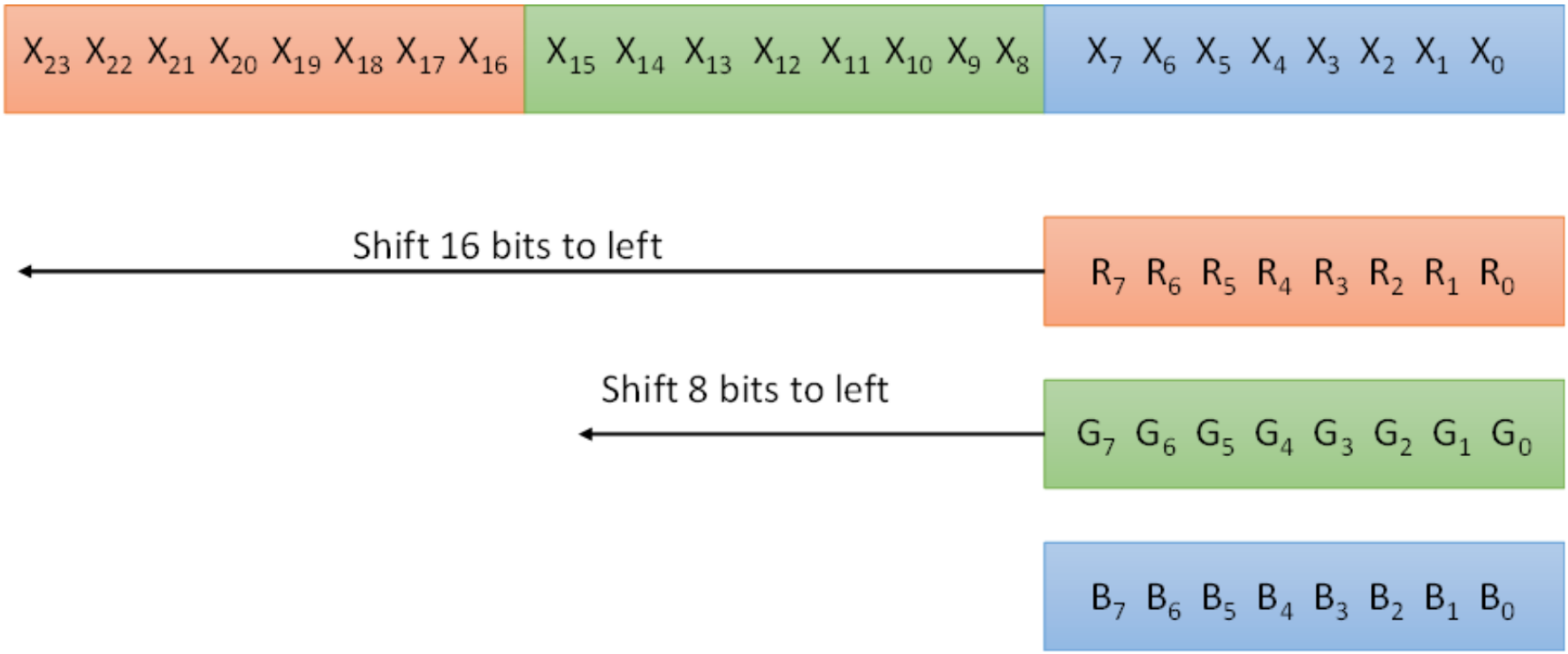


Predefined Colors

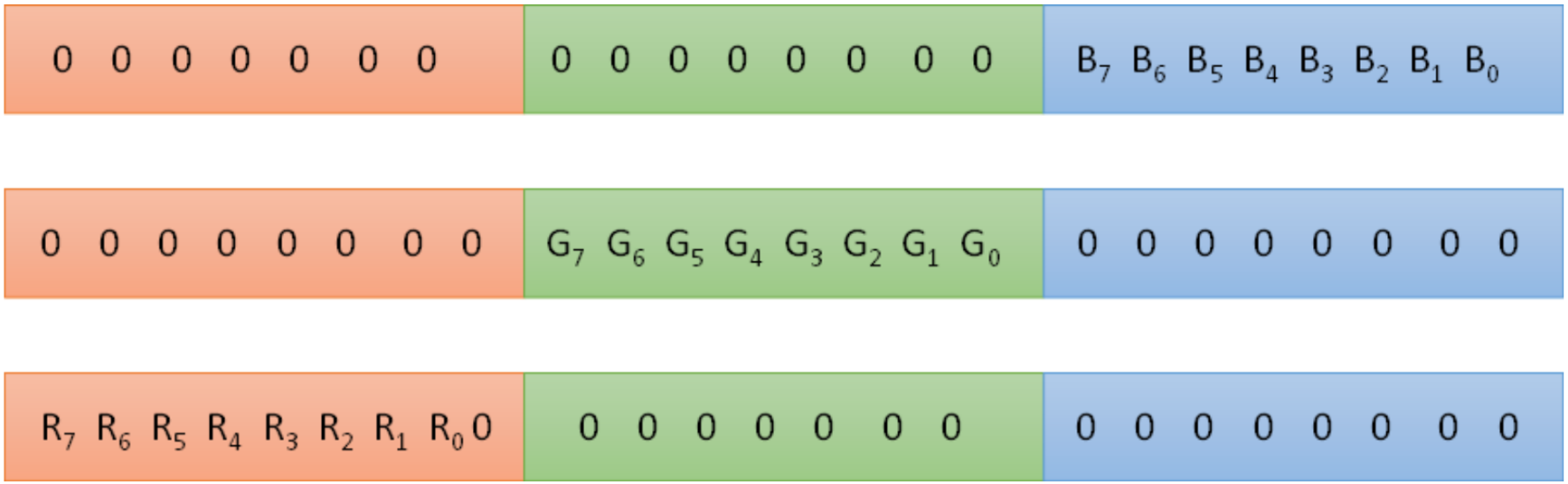
```
#define GRAPHICS_COLOR_CRIMSON                0x00DC143C
#define GRAPHICS_COLOR_CYAN                    0x0000FFFF
#define GRAPHICS_COLOR_DARK_BLUE              0x0000008B
#define GRAPHICS_COLOR_DARK_CYAN              0x00008B8B
#define GRAPHICS_COLOR_DARK_GOLDENROD          0x00B8860B
#define GRAPHICS_COLOR_DARK_GRAY              0x00A9A9A9
#define GRAPHICS_COLOR_DARK_GREEN              0x00006400
#define GRAPHICS_COLOR_DARK_KHAKI              0x00BDB76B
#define GRAPHICS_COLOR_DARK_MAGENTA             0x008B008B
#define GRAPHICS_COLOR_DARK_OLIVE_GREEN         0x00556B2F
#define GRAPHICS_COLOR_DARK_ORANGE             0x00FF8C00
#define GRAPHICS_COLOR_DARK_ORCHID             0x009932CC
#define GRAPHICS_COLOR_DARK_RED                 0x008B0000
#define GRAPHICS_COLOR_DARK_SALMON             0x00E9967A
#define GRAPHICS_COLOR_DARK_SEA_GREEN          0x008FBC8F
#define GRAPHICS_COLOR_DARK_SLATE_BLUE          0x00483D8B
#define GRAPHICS_COLOR_DARK_SLATE_GRAY          0x002F4F4F
```

Your Own Custom Colors

With bit manipulation you can make your own custom colors by mixing red, green and blue



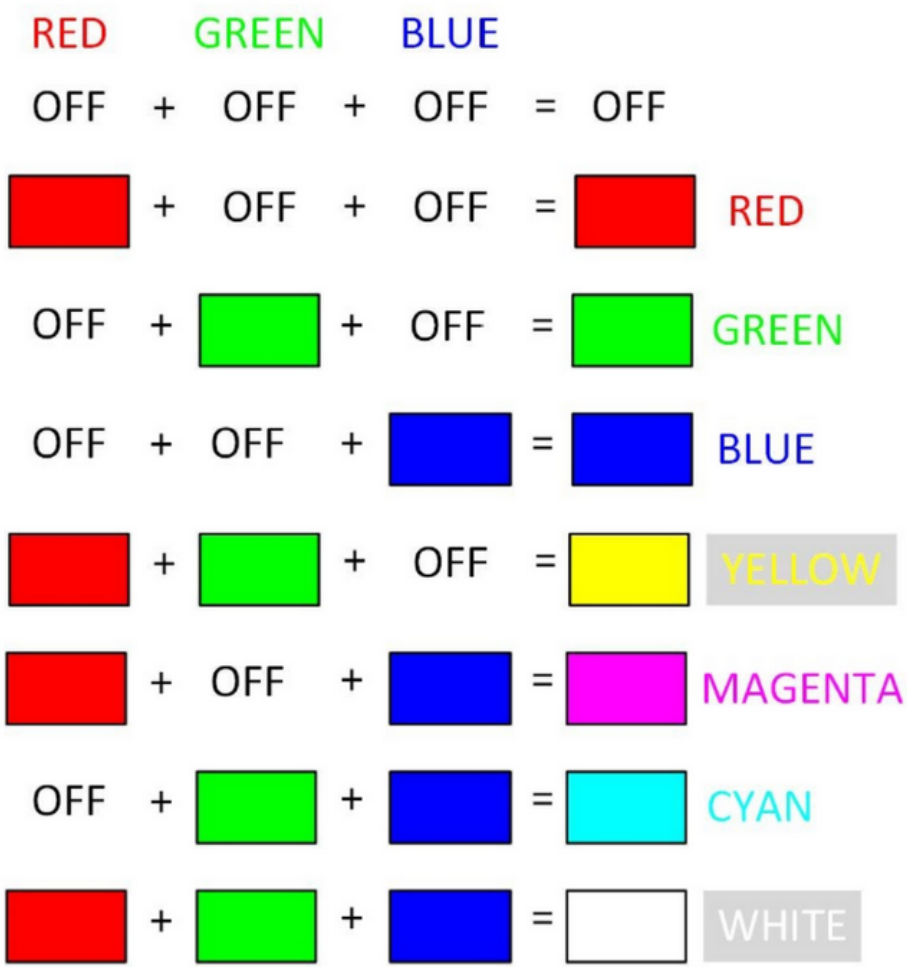
Your Own Custom Colors




```
char blue, red, green;
...
unsigned int color = ???; // What code is needed?
```

Tri-Color LED Comment

Color mixing for LEDs is also similar to this concept, but with less fidelity



Fonts

There are several fonts available in the graphics library:

- Cm (Computer Modern) (similar to Times New Roman)
- Cmsc (Computer Modern Small Caps)
- Cmss (Computer Modern Sans Serif) (similar to Helvetica/Arial)
- Cmtt (Computer Modern Teletype) (similar to `Courier New`)

- The fonts are also available as italic and bold and at various sizes
- The fonts take space and make your project bigger and the initial compile time longer
- Only include fonts that you need in your project
- The font files are located in the fonts folder right inside your project

Drawstring Function

The drawstring function is not well documented anywhere! The best guide you might find is probably on our own course website:

Drawstring Function documentation

Display Capabilities

- The background and foreground and inside a graphic context can be changed anytime throughout the program
- More than one graphic context can exist and the program can switch between them back and forth
- **Very important:** changing the colors or fonts in the graphic context does not change what is already drawn on the display. It changes the color of your "new" background and foreground and font and only when you draw new things you see the effect.
- It is relatively easy to draw any custom image on the display. You need to use the TI tool called image reformer to turn the image into a file that can be included and compiled with project.
- As you can literally see, the image is made of zeros and ones!

Example: Writing Text to the Display

Specifying the font:

`g_sFont<Type><Size>[b|i]`

Type:

- Type: `Cm`, `Cmsc`, `Cmss`, `Cmtt`
- Size: each font comes in sizes from 12pt to 48pt, in 2pt increments
- b – boldface, i – italics

Example: Writing Text to the Display

To draw text on the display use the following function:

```
Graphics_drawString(const Graphics_Context *context,
                    int8_t *string,
                    int32_t x, int32_t y, bool opaque);
```

- string is the pointer to the array of chars containing the text
- lLength is the number of characters from the string that are drawn; -1 will print the entire string
- x and y are the horizontal and vertical coordinates of where the string will be shown
- opaque
  - true – a rectangle in the background color is drawn first and then the text is drawn over that
  - false – only the text is drawn over the existing display content

Example: Writing Text to the Display

```
char* myString1 = "Hello1!";
char myString2[8] = "Hello2!";
...
Graphics_Context g_sContext;
Graphics_setFont(&g_sContext, &g_sFontCmtt24);
...
Graphics_drawString(&g_sContext, "Hello!", -1, 5, 20, true);
Graphics_drawString(&g_sContext, (int8_t*) myString1, -1, 5, 40, true);
Graphics_drawString(&g_sContext, (int8_t*) myString2, -1, 5, 60, true);
...
```

How to Print Variables that Contain Numerical Values?

- You must convert the numerical value to a string
- Two approaches
  - Convert each digit of the variable into a character
  - Use a buffer and the C function name `snprintf`

Converting Each Digit to a Character: Idea

```
\int myNumber = 123;
char myNumString[4] = "";

myNumString[2] = (myNumber % 10) + '0';
myNumString[1] = ((myNumber / 10) % 10) + '0';
myNumString[0] = ((myNumber / 100) % 10) + '0';
// now call Graphics_drawString passing (int8_t*) myNumString
```

Using a Buffer and snprintf

```
#define BUFFER_SIZE 10
char buffer[BUFFER_SIZE];
int myNum = 123; // number to print

snprintf(buffer, BUFFER_SIZE, "Hello! %03d", myNum);
// now call Graphics_drawString passing (int8_t*) buffer
```